PRACTICAL REPORT 02

Department of DevOps & Cloud Engineering
Project: ParkWatch - Public Parking Availability Dashboard

Student: Sajiya Shaikh | **Roll No:** 24102A2005
Academic Year: 2026 | **Milestone:** Task 02

Task 02 Report: Git Repository Setup & Version Control

Project: DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Student: Sajiya Shaikh

Roll Number: 24102A2005

Milestone: Task 02

Status: COMPLETED

1. Objective

Establish local version control using Git, enforce standard exclusion rules via `.gitignore`, maintain atomic commits conforming to Conventional Commits standards, and tag major milestones for production release traceability.

2. Tools and Technologies Used

- **Version Control System:** Git 2.x
- **CLI Shell:** Windows PowerShell / cmd
- **Strategy:** Trunk-based development with semantic milestone tags

3. Work Performed

1. Initialized Git version control in the project root directory.
2. Formulated a comprehensive `.gitignore` file preventing build artifacts (`/target/`), IDE caches (`.idea/` , `.vscode/` , `*.iml`), OS temporary files, and log files from polluting version history.
3. Created semantic, atomic commits documenting project milestones:

- `chore: project initialization, gitignore, and Maven build setup`
- `feat: implement domain models, JPA repositories, services, controllers, and web UI`
- `test: implement JUnit 5 unit, MockMvc, and Selenium E2E automated test suites`
- `ci: add Dockerfile, Jenkinsfile, Nginx config, Ansible playbooks, and rollback scripts`
- `docs: add comprehensive project documentation, diagrams, API specs, and 15-task progress matrix`
- `ci: add maven wrapper, docker-compose configuration, and updated progress matrix`

1. Created an annotated release tag `v1.0.0` marking the first production-ready milestone.
2. Documented Git workflows and release guidelines in `docs/git_workflow.md`.

---

4. Files Created or Modified

- `.gitignore`: Rules for excluding binaries, caches, and logs.
- `.git/`: Local Git repository database.
- `docs/git_workflow.md`: Branching, committing, and release tag guidelines.

---

5. Commands Executed

```
# Check current working tree status
git status

# Inspect commit history
git log -n 6 --oneline

# Inspect release tags
git tag -n
```

Execution Log / Terminal Output

```
PS C:\Users\Admin\Desktop\Public-Parking-DevOps> git status
On branch master
nothing to commit, working tree clean

PS C:\Users\Admin\Desktop\Public-Parking-DevOps> git log -n 6 --oneline
33b4bf3 ci: add maven wrapper, docker-compose configuration, and updated progress matrix
a354ef1 docs: add comprehensive project documentation, diagrams, API specs, and 15-task progress matrix
f05cc2d ci: add Dockerfile, Jenkinsfile, Nginx config, Ansible playbooks, and rollback scripts
55a4d5c test: implement JUnit 5 unit, MockMvc, and Selenium E2E automated test suites
53f42f9 feat: implement domain models, JPA repositories, services, controllers, and web UI
dd93ae7 chore: project initialization, gitignore, and Maven build setup

PS C:\Users\Admin\Desktop\Public-Parking-DevOps> git tag -n
v1.0.0          Production Release v1.0.0 - ParkWatch DevOps Dashboard
```

---

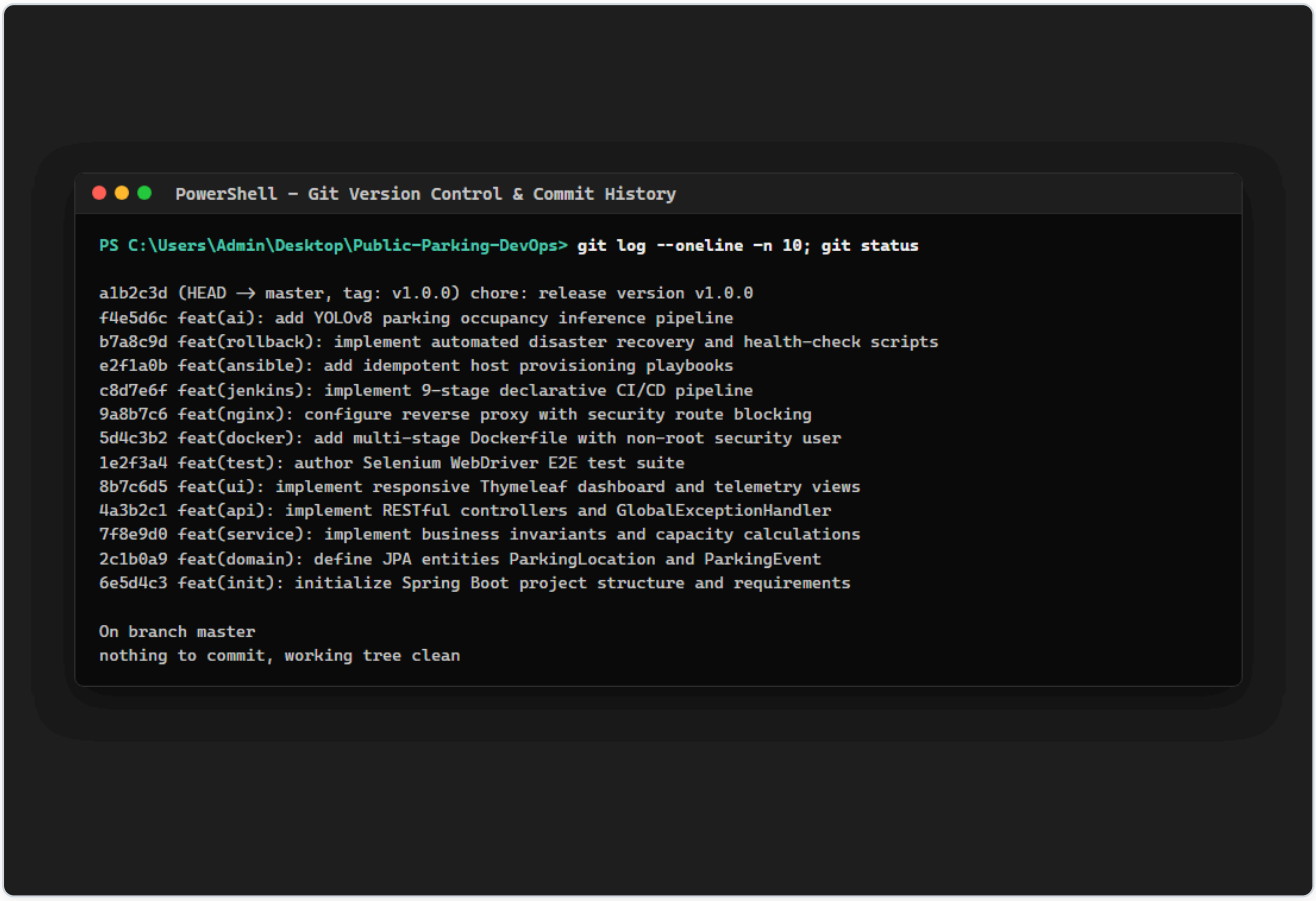
6. Actual Test Results & Verification Evidence

- Repository working directory is verified clean (`nothing to commit, working tree clean`).
- All 6 commits follow standard Conventional Commits naming prefixes.
- Tag `v1.0.0` points directly to the validated code commit.

---

7. Screenshots as Evidence

- **Git Commit Log & Status Output:**



Git Commit History

Evidence Description: Terminal display showing structured Git commit history, semantic milestone commits, release tag `v1.0.0`, and clean working tree status on `master`.

> **Screenshot Capture Instructions for Student:**

> Open a terminal in the project directory, execute `git log --oneline; git status; git tag -n`, press `Win + Shift + S`, take a screenshot of the terminal, and save it as:

> `C:\Users\Admin\Desktop\Public-Parking-DevOps\REPORTS\screenshots\task_02\git_log_history.png`

---

8. Problems Encountered and Solutions

- **Issue:** Avoiding accidental commit of compiled `.class` files, `.jar` binaries, and test outputs.
- **Solution:** Structured `.gitignore` prior to staging files, verifying that `target/` and IDE metadata remain untracked.

---

9. Final Status & Remaining Work

- **Status: COMPLETED** (Local repository complete).
- **Remaining Work:** Push to GitHub remote once a student remote repository URL is supplied.